

Software Design Plan

Motion Detection Application

Version 1

1. Purpose

The Version 1 Motion Detection Application is a local Python application that captures video from the system's default camera, detects meaningful visual movement between consecutive frames, and displays the motion state directly on the live camera feed.

The implementation shall remain strictly within the Version 1 architectural scope. It shall not perform object detection, object classification, object labeling, video recording, frame persistence, or network communication.

2. Technology Stack

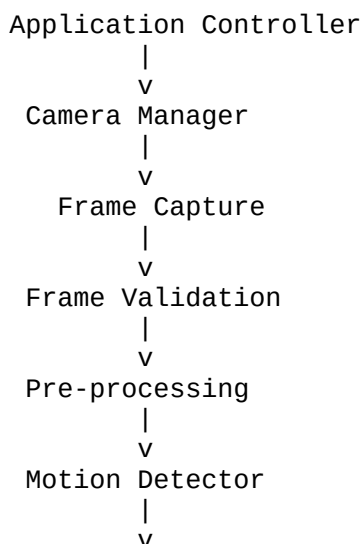
The implementation shall use:

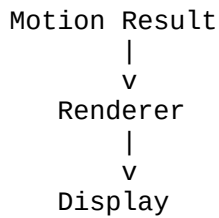
- Python
- OpenCV (cv2)
- NumPy
- Python Standard Library

OpenCV shall handle camera access, image processing, and display functionality. NumPy shall support frame and numerical operations. The Standard Library shall provide logging, timing, configuration support, and testing infrastructure where appropriate. No additional third-party functional dependencies are required.

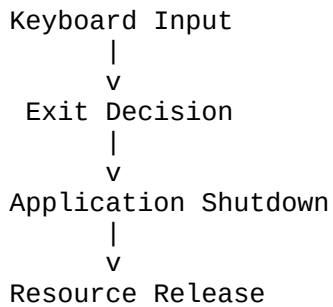
3. System Design

The software shall implement the following processing flow:





Keyboard input shall operate as a parallel control path:



This directly implements the modular architecture specified in the SAD.

4. Module Design

4.1 `main.py`

`main.py` shall be the application entry point and controller.

Responsibilities:

1. Load configuration.
2. Initialize logging.
3. Create the camera manager.
4. Open the default camera.
5. Verify camera availability.
6. Start the processing loop.
7. Coordinate frame processing.
8. Pass frames through preprocessing and motion detection.
9. Send results to the renderer.
10. Process keyboard input.
11. Initiate controlled shutdown.
12. Guarantee resource cleanup.

Conceptual structure:

```
def main():  
    initialize_logging()  
    camera = Camera()
```

```

try:
    camera.open()

    if not camera.is_open():
        handle_camera_failure()
        return

    run_processing_loop(camera)

except Exception:
    log_exception()

finally:
    camera.release()
    destroy_windows()

```

The controller shall contain application-flow logic rather than computer-vision algorithms.

4.2 camera.py

The Camera Manager shall encapsulate all direct camera interaction.

Proposed interface:

```

class Camera:
    def open(self) -> bool:
        ...

    def read(self):
        ...

    def is_open(self) -> bool:
        ...

    def release(self) -> None:
        ...

```

The implementation shall use OpenCV's camera interface, primarily `cv2.VideoCapture`.

The remainder of the application shall not directly access `cv2.VideoCapture`. This maintains low coupling and permits alternative camera implementations in later versions.

4.3 preprocessing.py

This module shall convert raw camera frames into a representation suitable for motion detection.

Initial processing:

Input: BGR camera frame

|
v

Convert to grayscale

|
v

Optional light smoothing

|

v

Processed frame

The module shall not determine whether motion occurred. Its responsibility ends after producing the processed frame.

Proposed interface:

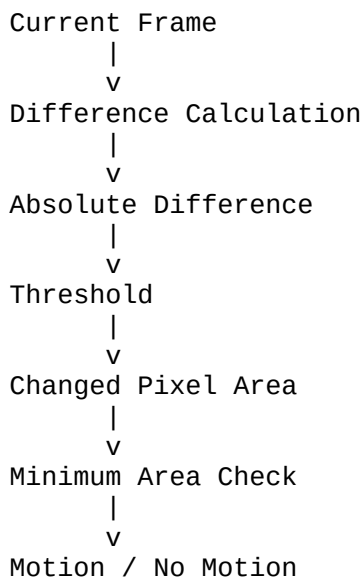
```
def preprocess(frame):  
    ...
```

The preprocessing stage must remain independent from the motion-detection algorithm so that preprocessing can evolve without changing the detector's interface.

4.4 **motion_detector.py**

The Motion Detector shall compare the current processed frame with the previous valid processed frame.

The initial algorithm shall be frame differencing:



Processing sequence:

1. Receive current processed frame.
2. Check whether a previous valid frame exists.
3. Calculate the absolute difference.
4. Apply the configured threshold.
5. Determine the amount of changed image area.
6. Compare the changed area against the minimum motion-area configuration.
7. Produce a `MotionResult`.
8. Store the current frame as the next valid previous frame.

The detector shall not attempt to determine what caused the movement.

A suitable result structure is:

```
@dataclass
class MotionResult:
    motion_detected: bool
    motion_area: int
```

`confidence` shall not be implemented in Version 1 because the SAD explicitly identifies it as unnecessary.

The first valid frame shall establish the baseline and should therefore produce `motion_detected = False`.

4.5 `renderer.py`

The `Renderer` shall display the processed result without containing detection logic.

Proposed interface:

```
def render(frame, motion_result):
    ...
```

Behavior:

```
motion_detected == True
    |
    +--> Display "Motion Detected"

motion_detected == False
    |
    +--> Display "No Motion"
```

The `renderer` shall also display the live frame.

Optional diagnostic information may be displayed when debug mode is enabled.

4.6 `config.py`

`config.py` shall contain centralized application configuration.

Initial configuration should include:

```
CAMERA_INDEX = 0

MOTION_THRESHOLD = ...
MIN_MOTION_AREA = ...

FRAME_WIDTH = 640
FRAME_HEIGHT = 480

EXIT_KEY = "q"

DEBUG = False
LOG_LEVEL = ...
WINDOW_NAME = "Motion Detection"
```

The exact numerical motion thresholds should be treated as implementation/test parameters rather than hard-coded into the detector.

Centralizing them allows threshold tuning without modifying processing code.

4.7 logger.py

The logging module shall provide a centralized logging configuration around Python's standard logging package.

Required levels:

ERROR
WARNING
INFO
DEBUG

Example events:

INFO:
 Camera initialized
 Application started
 Application shutdown

WARNING:
 Frame read failure
 Invalid frame

ERROR:
 Camera unavailable
 Display failure
 Unexpected application failure

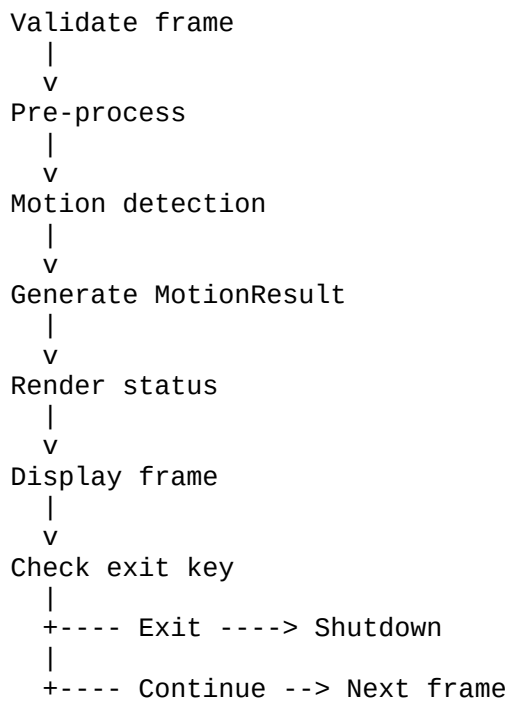
DEBUG:
 Frame captured
 Motion calculation
 Motion detected
 Motion not detected
 Motion area
 Resource release

Logs must contain diagnostic state only and must never contain raw camera frames or image data.

5. Main Processing Loop

The implementation shall follow this control flow:

```
START
|
v
Initialize configuration
|
v
Initialize logging
|
v
Open camera
|
+---- Failure ----> Log ERROR --> Shutdown
|
v
Capture frame
|
+---- Invalid ----> Log warning --> Next iteration
|
v
```



This follows the processing sequence defined by the architecture.

The previous frame shall only be updated after successful frame validation and preprocessing. An invalid frame must never overwrite a valid baseline.

6. Error Handling

Camera initialization failure

If the default camera cannot be opened:

1. Log an **ERROR**.
2. Display an appropriate failure message where possible.
3. Do not enter the processing loop.
4. Release any allocated resources.
5. Terminate cleanly.

Frame-read failure

If a frame cannot be read:

1. Log a **WARNING**.
2. Discard the failed frame.
3. Preserve the previous valid frame.
4. Continue processing when possible.

Invalid frame

Frames that are `None`, empty, or otherwise invalid shall not enter the motion-detection stage.

The detector shall never calculate motion using invalid data.

Display failure

If OpenCV display operations fail:

1. Log the failure.
2. Stop normal processing.
3. Execute the standard shutdown path.

Unexpected exception

The top-level controller shall catch unexpected exceptions, log the exception and stack trace, and execute resource cleanup.

Cleanup shall occur through a `finally`-style path so that camera and OpenCV resources are released even after an unexpected failure.

7. Resource Management

Resources shall have clear ownership.

```
main.py
|
+-- owns Camera
|
+-- owns application lifecycle
|
+-- owns shutdown

Camera
|
+-- owns VideoCapture

OpenCV display
|
+-- destroyed during shutdown
```

Normal shutdown and exceptional shutdown shall use the same cleanup mechanism.

At minimum:

```
camera.release()
cv2.destroyAllWindows()
```

The application shall not retain unnecessary frames after they are no longer required for comparison.

8. Configuration and State

The detector requires minimal persistent runtime state:

previous_frame
current_frame
motion_result

previous_frame shall represent only the most recent valid processed frame.

Configuration shall remain separate from runtime state.

This distinction prevents processing parameters from being accidentally modified during normal frame processing.

9. Testing Design

The SAD requires unit, integration, end-to-end, fault, and performance testing.

Unit tests

tests/test_motion_detector.py shall verify:

Identical frames	-> No Motion
Significant frame change	-> Motion
Small changes	-> No Motion
Threshold respected	-> Correct result
Empty frame	-> Safely rejected
Invalid frame	-> Safely rejected
First frame	-> No Motion

tests/test_preprocessing.py shall verify that:

- valid frames are processed;
- output is suitable for the detector;
- invalid inputs are handled safely.

tests/test_camera.py shall test the camera abstraction as far as practical without requiring a physical camera for every automated test.

Integration tests

Integration tests shall verify:

Camera

- > Frame Capture
- > Pre-processing
- > Motion Detector
- > Renderer
- > Display

The keyboard exit path and cleanup behavior shall also be tested.

End-to-end test

A basic E2E scenario shall be:

1. Start application.
2. Verify camera feed.

3. Keep scene stationary.
4. Verify "No Motion".
5. Introduce movement.
6. Verify "Motion Detected".
7. Stop movement.
8. Verify return to "No Motion".
9. Press configured exit key.
10. Verify clean shutdown.

These behaviors correspond directly to the Version 1 acceptance behavior in the SAD.

Fault tests

The implementation shall be tested with:

- unavailable camera;
- camera disconnection;
- invalid frames;
- dark scenes;
- noisy scenes;
- rapid movement;
- small movement;
- continuous movement;
- repeated exit-key presses.

Performance tests

The initial performance target shall be approximately:

- 640×480 processing resolution;
- 30 FPS or greater where hardware permits;
- no unnecessary frame storage;
- no unbounded memory growth;
- no continuously excessive CPU consumption.

Measurements should include FPS, frame-processing time, CPU usage, memory usage, and failed frame reads.

10. Privacy and Data Handling

All camera processing shall occur locally.

The implementation shall never:

- write frames to disk;
- write screenshots to disk;
- record video;
- upload frames;
- transmit camera data;
- connect to cloud services;
- use external APIs.

Camera frames shall exist only in application memory for the duration required for processing. Logs shall not contain image data.

11. Proposed Implementation Structure

```
motion_detection_app/  
|  
+-- main.py  
|  
+-- camera.py  
|  
+-- preprocessing.py  
|  
+-- motion_detector.py  
|  
+-- renderer.py  
|  
+-- config.py  
|  
+-- logger.py  
|  
+-- tests/  
|   |  
|   +-- test_motion_detector.py  
|   +-- test_preprocessing.py  
|   +-- test_camera.py
```

This is the file structure proposed by the architecture and keeps the Version 1 implementation small while maintaining module separation.

12. Module Dependency Rules

The implementation should enforce the following dependency direction:

```
main  
|  
+--> camera  
+--> preprocessing  
+--> motion_detector  
+--> renderer  
+--> config  
+--> logger  
  
motion_detector
```

```
|
+--> NumPy/OpenCV
|
X--> camera

renderer
|
+--> OpenCV
|
X--> motion_detector implementation
```

Most importantly, `motion_detector.py` shall not open or control the camera. It shall receive image data as input.

Similarly, `renderer.py` shall not determine whether motion exists. It shall receive the result and visualize it.

This preserves the SAD's requirements for single responsibility, low coupling, testability, configurability, and extensibility.

13. Implementation Sequence

Implementation should proceed in the following order:

```
1. config.py
   |
2. logger.py
   |
3. camera.py
   |
4. preprocessing.py
   |
5. motion_detector.py
   |
6. renderer.py
   |
7. main.py
   |
8. Unit tests
   |
9. Integration tests
   |
10. E2E and fault testing
    |
11. Performance validation
```

The motion detector and preprocessing modules should be developed and tested independently before integrating them with the physical camera.

14. Version 1 Acceptance Criteria

Implementation is complete when all of the following are satisfied:

- The default system camera opens successfully.
- A live camera feed is displayed.
- Meaningful visual movement is detected.

- "Motion Detected" is displayed during movement.
- "No Motion" is displayed when meaningful movement is absent.
- The application can be terminated through the configured keyboard command.
- Camera and OpenCV resources are released correctly.
- Invalid frames do not cause crashes or corrupt the previous valid frame.
- No frames or captures are saved.
- No network communication occurs.
- No object detection or object labeling occurs.
- Core motion-detection behavior has automated tests.
- Application failures generate useful diagnostic logs.
- The implementation remains modular enough for future camera and vision extensions.

These criteria align with the SAD's formal Version 1 Definition of Done.